# Tema 2 - Unix Timestamps

## Changelog:

- 23 Dec 2023 20:35: update schelet de cod typos
- 13 Ian 2023 23:55: adaugat teste pt taskurile 1-8 t2_tests_1-to-8.zip
- 14 Ian 2023 00:55: corectat testele pt taskul 8
- 15 Ian 2024 02:56: adaugat checker t2_checker.zip (thanks Cosmin & Mihnea & Teo pentru debugging la checker noaptea ca hotii)
- 15 Ian 2024 19:57: update checker + reparat teste task 9
- 15 Ian 2024 21:49: update teste sa fie timezone-urile de max 4 caractere

## Responsabili:

- Alin Popa

Termen de predare:

- Deadline soft: **Joi 11 Ianuarie 2024, 23:55**
- Deadline hard: **Duminica 14 Ianuarie 2024, 23:55**

Pentru fiecare zi (24 de ore) de întârziere, se vor scădea 10 puncte din nota acordată, până la atingerea deadline-ului hard.

## Întrebări

Dacă aveți nelămuriri, puteți să ne contactați pe forumul dedicat temei de casă nr. 2 sau pe canalul Temei 2. Nu se acceptă întrebări în ultimele 24 de ore înainte de deadline.

Atenție:

- Citiți cu atenție tot enunțul temei.

## Intro Unix Timestamps

Ne propunem să construim un sistem care simulează Epoch Converter.

Un Unix Timestamp este un mod standardizat de reprezentare a unui moment în timp, foarte folosit în multe aplicații, framework-uri, și sisteme de operare. Un Unix Timestamp este definit ca numărul de secunde care au trecut de la 1 ianuarie 1970, ora 00:00:00 (in timezone-ul UTC, fostul GMT, numit popular și "ora Angliei").

Exemplu:

- Timestamp-ul `0` corespunde datei `1 ianuarie 1970, ora 00:00:00`
- Timestamp-ul `3659` corespunde datei `1 ianuarie 1970, ora 01:00:59`
- Timestamp-ul `1703201916` corespunde datei `21 decembrie 2023, ora 23:38:36`

Pentru această temă nu aveți voie să folosiți funcții (din time.h sau din biblioteci 3rd party) care convertesc sau parsează date și timestamp-uri în mod automat.

## Schelet de cod

Pentru această temă trebuie să porniți de la scheletul de cod de aici: tema2_schelet.zip.

Atenție! **Nu** aveți voie să:

- redenumiți fișierele temei (timelib.c, bitpacking.c, planner.c)
- schimbați numărul, tipul, sau ordinea parametrilor funcțiilor din timelib.c și timelib.h
- schimbați sau redenumiți comenzile sau regulile din Makefile

## Cerință

### Partea A - DateTime Parsing

#### Task1 (5p) - Convert Unix Timestamp to Time

Implementați în fișierul `timelib.c` funcția:

```
TTime convertUnixTimestampToTime(unsigned int timestamp)
```

Funcția trebuie să returneze ora, minutul și secunda corespunzatoare timestamp-ului primit, sub forma unei structuri TTime:

```
typedef struct {
    unsigned char hour;
    unsigned char min;
    unsigned char sec;
} TTime
```

Exemple de rulare:

```
timestamp=0 => hour=0; min=0; sec=0
timestamp=3659 => hour=1; min=0; sec=59
timestamp=1703201916 => hour=23; min=38; sec=36
```

`Hour` ia valori între 0 și 23; `min` ia valori între 0 și 59; `sec` ia valori între 0 și 59.

#### Task2 (5p) - Convert Unix Timestamp to Date (1)

Implementați în fișierul `timelib.c` funcția:

```
TDate convertUnixTimestampToDateWithoutLeapYears(unsigned int timestamp)
```

Funcția trebuie să returneze ziua, luna și anul corespunzătoare timestamp-ului primit, sub forma unei structuri TDate:

```
typedef struct {
    unsigned char day;
    unsigned char month;
    unsigned int year;
} TDate
```

Exemple de rulare:

```
timestamp=0 => day=1; month=1; year=1970
timestamp=3659 => day=1; month=1; year=1970
timestamp=1703201916 => day=3; month=1; year=2024
```

`Day` ia valori între 1 și 31; `month` ia valori între 1 și 12; `year` ia valori de la 1970 în sus.

Pentru acest task, anii bisecți trebuie ignorați!

#### Task3 (5p) - Convert Unix Timestamp to Date (2)

Implementați în fișierul `timelib.c` funcția:

```
TDate convertUnixTimestampToDate(unsigned int timestamp)
```

Funcția trebuie să returneze ziua, luna și anul corespunzătoare timestamp-ului primit, sub forma unei structuri TDate ca la taskul anterior, însă luând în considerare anii bisecți. Un an bisect este un an cu 366 de zile (luna februarie are 29 de zile în loc de 28). Anii bisecți sunt toți anii divizibili cu 4 (e.g. 2004, 2008, 2012) cu excepția anilor care sunt multipli de 100 dar nu și multipli de 400 (exemplu: anii 2000 și 2400 sunt bisecți, dar anii 2100, 2200, 2300 nu sunt).

Exemple de rulare:

```
timestamp=0 => day=1; month=1; year=1970
timestamp=3659 => day=1; month=1; year=1970
timestamp=1703201916 => day=21; month=12; year=2023
```

Pentru task-urile de la 3 în sus, anii bisecți trebuie luați în considerare!

#### Task4 (5p) - Unix Timestamp with Timezones

Implementați în fișierul `timelib.c` funcția:

```
TDateTimeTZ convertUnixTimestampToDateTimeTZ(unsigned int timestamp, TTimezone *timezones, int timezo
```

Funcția primește un Unix Timestamp, un vector de timezone-uri, și un index care definește timezone-ul din vector ce trebuie folosit. Un timezone (fus orar) este reprezentat de o structură TTimezone, cu următoarele componente:

- name: numele timezone-ului (e.g.: "UTC", "GMT", "EET", "EST")
- utc_hour_difference: diferența în număr de ore față de timezone-ul UTC (în care se consideră că este orice Unix Timestamp).

```
typedef struct {
    char name[5];
    signed char utc_hour_difference;
} TTimezone
```

Funcția trebuie să returneze o structură TDateTimeTZ, cu următoarele componente:

- date: data corespunzătoare timestamp-ului primit
- time: ora, minutul, secunda corespunzătoare timestamp-ului primit
- tz: pointer la timezone-ul corespunzător din vectorul de timezone-uri

```
typedef struct {
    TDate date;
    TTime time;
    TTimezone *tz;
} TDateTimeTZ
```

Pentru această funcție, la conversia din Unix Timestamp în formatul Date+Time, trebuie luată în considerare diferența de timp impusă de timezone. Spre exemplu, timestamp-ul `1703201916` corespunde datei `21 decembrie 2023, ora 23:38:36` dacă acesta este considerat în timestamp-ul UTC (echivalent cu GMT), pentru care `utc_hour_difference` este 0. Însă același timestamp, într-un timezone în care `utc_hour_difference` este 2 (spre exemplu, EET) va corespunde datei `22 decembrie 2023, ora 01:38:36` (2 ore mai târziu). Același timestamp, într-un timezone în care `utc_hour_difference` este -5, va corespunde datei `21 decembrie 2023, ora 18:38:36`.

Fie următorul vector de timezone-uri: `[("UTC", 0); ("EET", 2); ("PST", -8)]`

Exemple de rulare:

```
timestamp=1703201916, timezone_index=0 => 21-12-2023 23:38:36
timestamp=1703201916, timezone_index=1 => 22-12-2023 01:38:36
timestamp=1703201916, timezone_index=2 => 21-12-2023 15:38:36
```

Se garantează pentru toate timezone-urile că: `-13 < utc_hour_difference < 15`

Timestamp-urile de la taskurile 1-3 se consideră în timezone-ul standard UTC. Practic, prin conversia unui Unix Timestamp la formatul dată+timp așa cum am făcut la taskurile precedente, obținem data și ora corespunzătoare timezone-ului UTC+0. Schimbarea la acest task este că trebuie să luăm în considerare diferența cu un anumit număr de ore (în plus sau în minus) dată de timezone-ul cerut.

#### Task5 (5p) - DateTimeTz back to Unix Timestamp

Implementați în fișierul `timelib.c` funcția:

```
unsigned int convertDateTimeTZToUnixTimestamp(TDateTimeTZ)
```

Funcția primește un TDateTimeTZ așa cum a fost definit mai sus și execută operația inversă, calculând Unix Timestamp-ul corespunzător.

Unix Timestamp-ul trebuie calculat așa cum este el în timezone-ul UTC! Spre exemplu, `22 decembrie 2023, ora 01:38:36` în timezone EET (UTC+2) va corespunde timestamp-ului `1703201916`.

Fie următorul vector de timezone-uri: `[("UTC", 0); ("EET", 2); ("PST", -8)]`

Exemple de rulare:

```
21-12-2023 23:38:36 (UTC) => 1703201916
22-12-2023 01:38:36 (EET) => 1703201916
21-12-2023 15:38:36 (PST) => 1703201916
```

#### Task6 (5p) - Printing DateTimes

Implementați în fișierul `timelib.c` funcția:

```
void printDateTimeTZ(TDateTimeTZ datetimetz)
```

Funcția afișează la stdout datetime-ul primit, în următorul format: `[data] [luna (cuvânt)] [anul], [ora]:[minutul]:[secunda] [timezone name] (UTC[+/-][timezone utc hour diff])`

Exemple de afișare:

```
1 ianuarie 1970, 05:00:00 UTC (UTC+0)
22 decembrie 2023, 01:38:36 EET (UTC+2)
21 decembrie 2023, 15:38:36 PST (UTC-8)
```

### Partea B - Network Packing

Pentru partea B, toate datele se consideră în timezone UTC.

#### Task7 (10p) - Basic Network Packing

Printr-o rețea de calculatoare se trimite un vector de obiecte de tip "dată" (zi, lună, an). Din motive de economisire a spațiului, aceste date sunt memorate pe biții unui unsigned int (presupunem unsigned int de 32 de biți), în următorul format:

| Biți nefolosiți | Anul | Luna | Ziua |
|---|---|---|---|
| 17 biți | 6 biți | 4 biți | 5 biți |

Astfel, în cadrul unui unsigned int:

- cei mai **nesemnificativi** 5 biți memorează ziua, sub forma unui număr între 1 (`00001`) și 31 (`11111`)
- următorii 4 biți memorează luna, sub forma unui număr între 1 (`0001`) și 12 (`1100`)
- următorii 6 biți memorează anul ca diferență față de 1970, sub forma unui număr între 0 (`000000`) ce reprezintă anul 1970, și 63 (`111111`) ce reprezintă anul 2033

Scrieți un program (în fișierul `bitpacking.c`) care citește:

- numărul taskului (pentru acest task va fi 7)
- un număr natural N
- N numere de tip unsigned int ce reprezintă fiecare câte o dată memorată pe biți așa cum este definit mai sus.

Programul trebuie să afișeze la stdout datele citite, câte una pe linie, **sortate în ordine cronologică**, în formatul `[zi] [lună] [an]` (exemplu: `21 decembrie 2023`).

#### Task8 (20p) - Complex Network Packing

Modificați programul din fișierul `bitpacking.c` pentru a rezolva acest task atunci când se citește 8 pentru numărul taskului.

O problemă majoră a packing-ului pe biți așa cum a fost definit la taskul 7 este că nu e foarte eficient. Deși sunt necesari doar 15 biți (6+4+5) pentru memorarea unei date, sunt folosiți cei 32 de biți ai unui unsigned int. Pentru a rezolva această problemă, vom împacheta datele la un loc folosind aceleași valori unsigned int. Astfel:

- prima dată se va regăsi în cei 15 cei mai **nesemnificativi** biți (0-14) din primul unsigned int citit
- a doua dată se va regăsi în următorii 15 biți ai primului unsigned int (ziua în biții 15-19, luna în biții 20-23, anul în biții 24-29)
- a treia dată se va regăsi în ultimii 2 biți (cei mai **semnificativi**) ai primului unsigned int și primii 13 biți ai celui de-al doilea unsigned int (ziua în biții 30-31 ai primului int și biții 0-3 ai următorului int; luna în biții 4-7, iar anul în biții 8-13 ai celui de-al doilea int)
- și asa mai departe.

Pentru un plus de realism, dorim să simulăm și cazul în care pe rețea există perturbații care alterează datele transmise. Pentru asta, vom avea câte un bit de control (separat de împachetarea descrisă mai sus) pentru fiecare unsigned int citit. Acest bit de control va fi definit ca `B1%2`, unde `B1` reprezintă numărul de biți de 1 din unsigned int-ul respectiv. Spre exemplu:

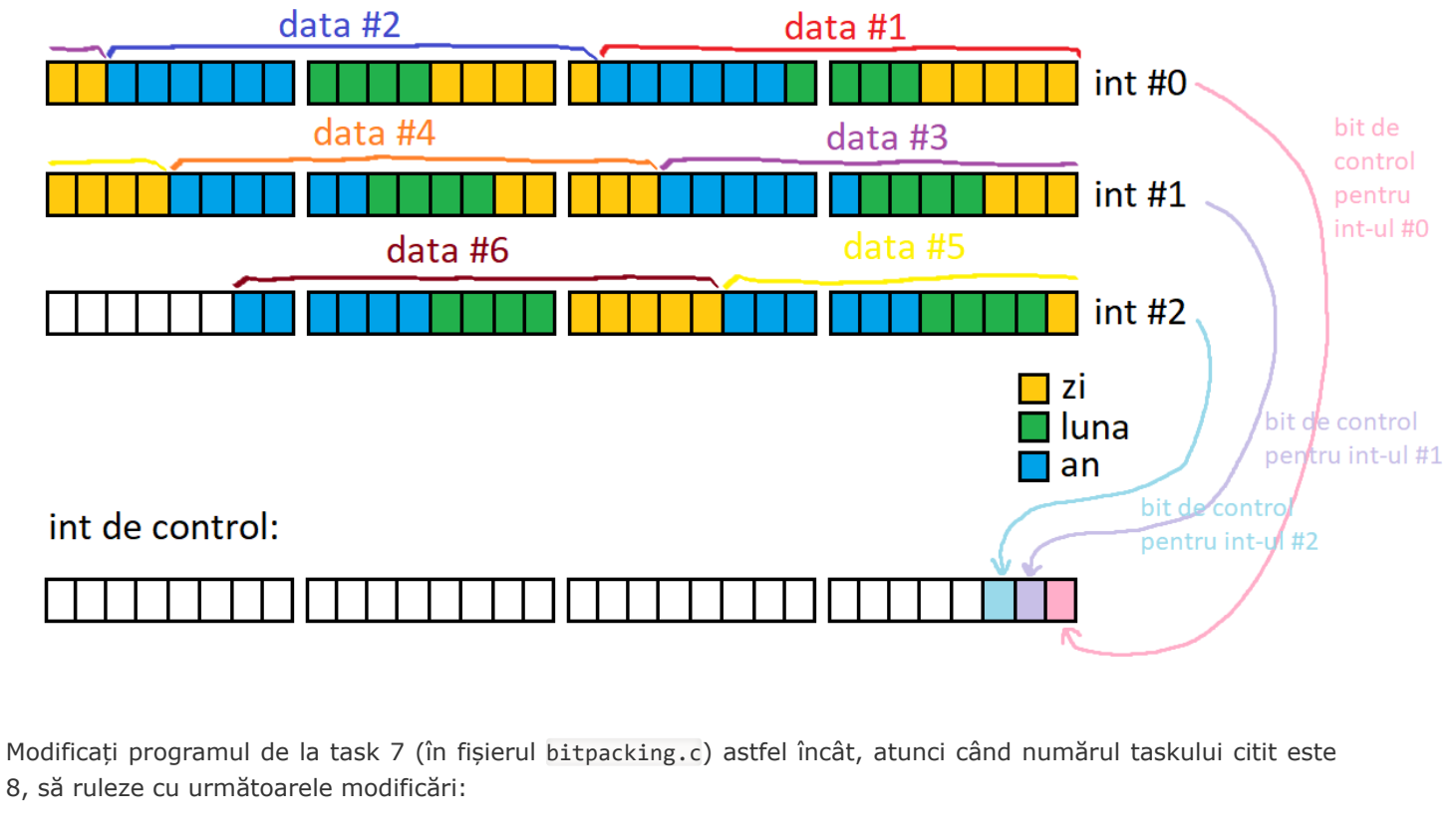
- pentru unsigned int-ul `00000000 00000000 00000000 00000000`, bitul de control va fi 0 (sunt 0x biți egali cu 1 în acest int)
- pentru unsigned int-ul `01010101 01010101 01010101 01010101`, bitul de control va fi 0 (sunt 16x biți egali cu 1 în acest int, iar 16 este număr par - `16%2 == 0`)
- pentru unsigned int-ul `01010101 01010101 01010101 01010111`, bitul de control va fi 1 (sunt 17x biți egali cu 1 în acest int, iar 17 este număr impar - `17%2 == 1`)

Biții de control vor fi și ei împachetați într-un șir de unsigned int-uri de control. Astfel:

- cel mai nesemnificativ bit din primul unsigned int de control reprezintă bitul de control pentru primul unsigned int citit
- următorul bit din primul unsigned int de control reprezintă bitul de control pentru al doilea int citit
- etc.

Biții de control vor fi folosiți astfel:

1. pentru fiecare unsigned int de date citit, se numără biții egali cu 1 din acesta (`B1`) și se calculează `B1%2`
2. se compară această valoare calculată cu bitul de control citit de la stdin
3. dacă cele două valori coincid, se consideră că acest unsigned int de date a fost recepționat corect de pe rețea
4. dacă cele două valori nu coincid, se consideră că acest unsigned int a fost corrupted, prin urmare toate datele ai căror biți fac parte din acest unsigned int vor fi ignorate și nu vor fi afișate în outputul programului.



Modificați programul de la task 7 (în fișierul `bitpacking.c`) astfel încât, atunci când numărul taskului citit este 8, să ruleze cu următoarele modificări:

- citește un număr natural N, ca mai devreme, ce reprezintă numărul de date (atenție - numărul de date, nu de int-uri) care trebuie citite
- citește un șir de unsigned int-uri ce conțin aceste date împachetate după descrierea de mai sus, până se citesc toate cele N date
- citește un șir de unsigned int-uri de control până se află valoarea bitului de control pentru toate unsigned int-urile de date citite

Programul trebuie să afișeze la stdout datele citite **(dar numai cele ai căror biți sunt în totalitate numai în unsigned int-uri care nu sunt corupte)**, câte una pe linie, sortate în ordine cronologică, în același format ca la taskul 7.

### Partea C - Google Calendar

#### Task9 (35p) - Multi-timezone event planner

Ne propunem să creăm un sistem de management de evenimente care găsește automat un interval de timp în care persoanele ce trebuie să participe la eveniment sunt libere.

Scrieți un program (în fișierul `planner.c`) care:

- citește un număr T ce reprezintă numărul de timezone-uri din program
- citește fiecare timezone, câte unul pe linie:
  - numele timezone-ului (un string de maxim 4 caractere), și diferența în ore față de UTC
- citește un număr P ce reprezintă numărul de persoane
- pentru fiecare persoană, citește următoarele:
  - numele persoanei (un cuvânt de maxim 10 caractere)
  - numele timezone-ului în care această persoană locuiește
  - un număr K ce reprezintă numărul de intervale în care această persoană este liberă
  - citește fiecare interval, dat de următoarele informații:
    - anul, luna, ziua, ora când intervalul liber începe (datele sunt considerate în timezone-ul persoanei careia îi aparțin)
    - durata intervalului liber în număr de ore
- citește parametrii evenimentului:
  - un număr întreg F (F mai mic sau egal cu P) ce reprezintă numărul minim de persoane care trebuie să participe la eveniment
  - durata evenimentului în număr de ore

Programul trebuie să găsească **primul** interval temporal în care evenimentul se poate organiza astfel încât minim F persoane să fie disponibile pe întreaga durată a evenimentului. Dacă nu există niciun interval în care să se poată organiza evenimentul, programul trebuie să afișeze "imposibil". Dacă se găsește un interval în care evenimentul se poate ține, atunci programul trebuie să afișeze P linii (câte una pentru fiecare dintre cele P persoane, sortate alfabetic după nume), cu următorul conținut:

- dacă persoana respectivă poate ajunge la eveniment, se afișează `[nume]: [data]` (data fiind reprezentată sub forma an, lună, zi, oră, minut, secundă, timezone - în același format ca la taskul 6) evenimentului, în timezone-ul acestei persoane
- dacă persoana respectivă nu va fi disponibilă pentru eveniment, se afișează "invalid"

Exemplu de rulare:

```
1
UTC 0

3

Alin UTC 1
2023 12 20 12 5

Vlad UTC 3
2023 12 20 12 1
2023 12 20 14 2
2023 12 20 16 1

Traian UTC 1
2023 12 20 12 1

2 3
```

Pentru inputul de mai sus, outputul trebuie să fie:

```
Alin: 20 decembrie 2023, 14:00:00 UTC (UTC+0)
Traian: invalid
Vlad: 20 decembrie 2023, 14:00:00 UTC (UTC+0)
```

Explicație:

- Avem un singur timezone, și anume UTC (UTC+0)
- Avem 3 persoane: Alin, Vlad, Traian (toate în același timezone)
  - Toate intervalele persoanelor sunt în acest caz în aceeași zi (20 decembrie 2023)
  - Alin este liber între orele 12-17
  - Vlad este liber între orele 12-13, 14-17 (14-16 și 16-17 sunt două intervale lipite)
  - Traian este liber între orele 12-13
- Evenimentul necesită cel puțin 2 persoane libere și durează 3 ore
- Singurul interval în care se poate organiza este 14-17 (participă Alin și Vlad)
- Persoanele sunt afișate în ordine alfabetică

Încă un exemplu de input pentru care rezultatul va fi același, dar de data asta persoanele se află în timezone-uri diferite:

```
3
UTC 0
EET 2
PST -8

3

Alin UTC 1
2023 12 20 12 5

Vlad EET 3
2023 12 20 14 1
2023 12 20 16 2
2023 12 20 18 1

Traian PST 1
2023 12 20 4 1

2 3
```

- Anul va fi dat sub forma unui număr între 1970-2050
- Luna va fi un număr între 1-12
- Ziua va fi un număr între 1-31
- Ora va fi un număr între 0-23
- Se consideră că toate intervalele încep la ore fixe (minutul și secunda se consideră 00:00)

Toate datele citite sunt considerate în timezone-ul perosoanei căreia îi aparțin. Spre exemplu, dacă persoana A (UTC+2) este liberă pe 20 decembrie 2023 începând cu ora 14 timp de 1 ora, iar persoana B (UTC+1) este liberă pe 20 decembrie 2023 începând cu ora 13 timp de 1 ora, cele două persoane sunt de fapt libere în același timp (deoarece ora 13 UTC+1 este ora 14 UTC+2). Hint: cele două date se vor converti la același Unix Timestamp.

Hint: folosiți funcțiile definite la partea A.

#### Task10 (5p) - Clean Valgrind

Pentru acest task, trebuie să aveți punctajul maxim pe taskul 9 și să nu aveți memory leaks la verificarea folosind utilitarul valgrind pe acesta.

Utilitarul se va rula folosind urmatoarea comanda:

```
valgrind --tool=memcheck --leak-check=full --error-exitcode=1 ./planner
```

## Coding Style

La fel ca la temele precedente, există o depunctare de până la -20p pentru coding style. Checkerul verifică coding style-ul în mod automat.

## Validare locală temă

**Checker: t2_checker.zip**

## Trimitere temă

Tema va fi trimisă folosind Moodle, cursul **Programarea Calculatoarelor (CB & CD)**, activitatea "Tema 2".

Se va posta un anunț pe forum si Teams când se va deschide upload-ul.

Toate temele sunt testate în mod automat pe Moodle.

Arhiva temei se va încărca folosind formularul de submisie (butonul **Add submission**.

Rezultatele vor fi disponibile în secțiunea **Feedback** - nota apare la linia **Grade**, iar outputul checkerului și erorile apar la sectiunea **Feedback comments**. Dacă apare un buton albastru în formă de plus, trebuie să dați click pe el pentru a afișa întregul output al checkerului.
**Citiți cu atenție** informațiile afișate în **Feedback** pentru a vă asigura că tema a fost rulată cu succes; o eroare comună este dată de faptul că conținutul arhivei nu respectă structura dorită (ex. fișierele sunt într-un alt director).

**Punctajul final al temei** este afișat la linia **Grade** și la finalul outputului din checker.

Conținutul arhivei trebuie să fie următorul:

1. Fișierele **timelib.c**, **timelib.h**
2. Fișierul **bitpacking.c**
3. Fișierul **planner.c**
4. Fișierul **Makefile**
5. Un fișier README în care descrieți rezolvarea temei.

Arhiva trebuie să fie de tipul **zip**.

Nu includeti fisierele checkerului in arhiva voastra.

In cazul in care testele va trec local, insa pica pe vmchecker cel mai probabil aveti o sursa de "undefined behavior in cod". Pentru a va asigura ca scapati de aceste probleme, compilati cu flagul de compilare `-Wall` si rezolvati toate warning-urile.

### Listă depunctări

Lista nu este exhaustivă.

- O temă care nu compilează și nu a rulat pe **vmchecker** nu va fi luată în considerare
- O temă care nu rezolvă cerința și trece testele prin alte mijloace nu va fi luată în considerare
- **NU acceptăm teme copiate**. În cazul unei teme copiate se scade punctajul aferent temei din punctajul total.
- [-20.0]: Nerezolvarea tuturor erorilor și warningurilor de coding style

Search

**Resurse generale**
- Anunțuri CB/CD
- Regulament: seria CA
- Regulament: seria CB/CD
- Catalog 2023-2024 CB/CD
- Calendar

**Tutoriale**
- Good Practices
- Coding style example
- Debugging
- Read documentation
- Code understanding
- Logging
- Unit tests

**Resurse laborator**
- VM - CA
- Test practic - CA
- Exerciții laborator - CA
- Checker laborator CB/CD
- WSL Setup
- Culegere de Probleme

**Cursuri**
Continutul Tematic

**Laboratoare**
01. Unelte de programare
02. Tipuri de date. Operatori.
03. Instrucțiunile limbajului C
04. Funcții
05. Tablouri. Particularizare - vectori
06. Matrice. Operaţii cu matrice
07. Optimizarea programelor folosind operaţii pe biţi
08. Pointeri. Abordarea lucrului cu tablouri folosind pointeri
09. Alocarea dinamică a memoriei. Aplicaţii folosind tablouri şi matrice
10. Prelucrarea şirurilor de caractere. Funcţii. Aplicaţii
11. Structuri. Uniuni. Aplicaţie: Matrice rare
12. Operaţii cu fişiere. Aplicaţii folosind fişiere.
13. Parametrii liniei de comandă. Preprocesorul. Funcţii cu număr variabil de parametri
14. Recapitulare

**Teme de casă (general)**
- Indicații generale

**Teme de casă: seria CA**

**Teme de casă: seria CB/CD**
- Tema 0
- Tema 1
- Tema 2

**Table of Contents**

- Tema 2 - Unix Timestamps
  - Changelog:
  - Responsabili:
  - Întrebări
  - Intro Unix Timestamps
  - Schelet de cod
  - Cerință
    - Partea A - DateTime Parsing
      - Task1 (5p) - Convert Unix Timestamp to Time
      - Task2 (5p) - Convert Unix Timestamp to Date (1)
      - Task3 (5p) - Convert Unix Timestamp to Date (2)
      - Task4 (5p) - Unix Timestamp with Timezones
      - Task5 (5p) - DateTimeTz back to Unix Timestamp
      - Task6 (5p) - Printing DateTimes
    - Partea B - Network Packing
      - Task7 (10p) - Basic Network Packing
      - Task8 (20p) - Complex Network Packing
    - Partea C - Google Calendar
      - Task9 (35p) - Multi-timezone event planner
      - Task10 (5p) - Clean Valgrind
    - Coding Style
    - Validare locală temă
    - Trimitere temă
      - Listă depunctări

CC BY-SA CHIMERIC DE W3C CSS DOKUWIKI GET FIREFOX RSS XML FEED W3C XHTML 1.0